TIPE: Les codes de Reed-Solomon : principe et utilisation

Matéo Mangialomini 33497

TIPE

2022-2023

Sommaire

1. Introduction
2. Théorie de Galois
3. Code correcteur de Reed-Solomon
4. Performances et optimisations
5. Conclusion

Introduction

Lien avec le thème : la ville

- Communication entre les feux de signalisation urbains

Introduction

Corruption lors du transfert de données

Envoyeur	$\longrightarrow$	Canal de transmission	$\longrightarrow$	Receveur
Message envoyé 0101101	$\longrightarrow$	Retournement(s) de bit(s)	$\longrightarrow$	Message reçu 0101010

Introduction

Principe des codes correcteurs

Envoyeur	$\longrightarrow$	Canal de transmission	$\longrightarrow$	Receveur
Message envoyé 0101101**10**	$\longrightarrow$	Retournement(s) de bit(s)	$\longrightarrow$	Message reçu 0**1**0**1**0**10**

Receveur	$\longrightarrow$	Code correcteur	$\longrightarrow$	Receveur
Message corrompu 0**1**0**1**0**10**	$\longrightarrow$	Reed-Solomon	$\longrightarrow$	Message réparé 0**1**0**1**0**10**

$\underbrace{10}$

bits de redondance

Théorie de Galois

- "Galois Field", noté $\mathbb{F}_p$ = Corps de cardinal fini
  p cardinal
- Parallèle avec $\mathbb{Z}/n\mathbb{Z}$
  $\longrightarrow$ corps si et seulement si n premier
- Propriété importante
  $\longrightarrow$ puissance de nombres premiers et donc de 2 !

Théorie de Galois

Lien avec les codes de Reed-Solomon

- Messages représentés sur $\mathbb{F}_2 \longrightarrow$ binaire
- Groupement par symboles $\longrightarrow$ 8 bits = 1 octet
- 10101010 10101101 01110101 00000101 01011001 10101010

0	1	1	1	1	0
1	0	0	0	0	1
0	1	1	1	0	0
1	1	0	0	1	1
0	0	1	0	1	0
1	1	1	0	0	1
0	0	1	0	1	0
1	1	0	0	0	1

- Intérêt $\rightarrow$ structurer les données

Paramètres d'un code de Reed-Solomon(n,k)

- n : longueur du code
- k : longueur du message
- 2t : symboles ajoutés
- $n - k = 2t$ $(= 2^p - 1)$

Limite théorique

- Majoration de Singleton :
  $2 * erreurs + effacements \leqslant n - k = 2t$
  $\longrightarrow$ effacements = erreurs dont on connait la position

Interprétation de $\mathbb{F}_{2^8}$

- Symbole = Caractère

Binaire	Polynôme	Exponentielle	*Numérique*
000011101	$x^4 + x^3 + x^2 + 1$	α^i	*29*

- Symboles = Message

"bonjour"
98,111,110,106,111,117,114
$98x^6 + 111x^5 + 110x^4 + 106x^3 + 111x^2 + 117x + 114$
$1100010x^6 + 1101111x^5 + 1101110x^4 + ...$

Polynôme générateur

- α élément générateur de $(\mathbb{F}_p, *)$
- $G(x) = \prod_{i=1}^{2t} (x - \alpha^i)$

Encodage

- Polynôme Message : $M(x)$
- $M(x) = G(x)Q(x) + R(x)$
- Polynôme envoyé : $C(x) = M(x) + R(x) = G(x)Q(x)$
  $\longrightarrow$ dans $\mathbb{F}_2$: $+ = \oplus =$ -

Polynôme d'erreur

- Polynôme reçu : $A(x)$
- $E(x) = C(x) - A(x) = \sum_{j=1}^{e} E_{i_j} x^{i_j}$

  hypothèse : $e \in [[1, t]]$

Polynôme des syndrômes

- $\forall k \in [[1, 2t]], C(\alpha^k) = 0$
  donc $A(\alpha^k) = C(\alpha^k) + E(\alpha^k) = E(\alpha^k)$
- $S(x) = \sum_{k=1}^{2t} E(\alpha^k) x^k$

Équation clé

- $\Lambda(x) = \prod_{k=1}^{e} (1 - \alpha^{i_k} x)$
  Polynôme localisateur d'erreurs
- $\Omega(x) = \sum_{k=1}^{e} E_{i_j} \alpha^{i_k} \prod_{l=1, l \neq k}^{e} (1 - \alpha^{i_l} x)$
  Polynôme évaluateur d'erreurs
- $\Lambda(x) S(x) \equiv \Omega(x) [x^{2t}]$
  $\Leftrightarrow$ Système à 2t équations et 2t inconnues $\Leftrightarrow$ système matriciel

Décodage

Algorithm 1 Décodage

1: $S(x) \leftarrow$ CalculSyndromes(A(x))
2: $\Lambda(x) \leftarrow$ Berlekamp-Massey$(A(x), S(x))$
3: $\Omega(x) \leftarrow$ Reste $S(x)\Lambda(x)/x^{2t}$ (équation clé)
4: $(i_1, ...i_e) \leftarrow$ PositionsRacinesForceBrute$(\Lambda(x))$
5: $(E_{i_1}, ..., E_{i_e}) \leftarrow$ Forney$(\Omega(x), \Lambda(x))$
6: **renvoyer** $E(x) + A(x)$

Berlekamp-Massey

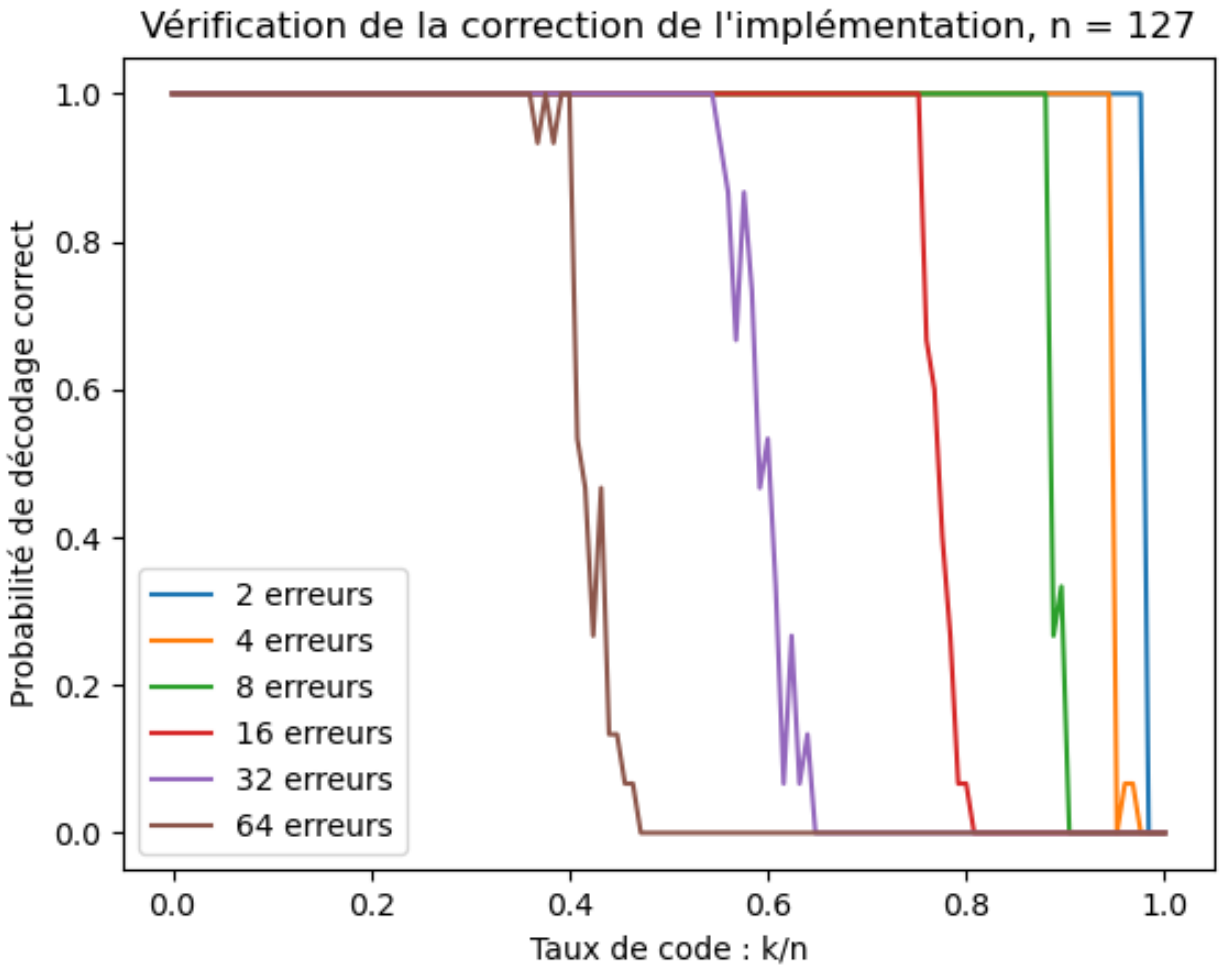
Algorithm 2 Berlekamp-Massey

$L, r \leftarrow 0$
$ancien\Lambda(x), \Lambda(x) \leftarrow 1$
for $i = 0, degre(S(x)) = 2t$ **do**
    $\Delta \leftarrow \sum_{j=1}^{n-1} \Lambda_j S_{r-j}$
    if $\Delta \neq 0$ **then**
        if $2L \leqslant r - 1$ **then**
            $ancien\Lambda(x), \Lambda(x) \leftarrow \Lambda(x) * \Delta^{-1}, ancien\Lambda(x) * \Delta$
        end if
        $\Lambda(x) \leftarrow \Lambda(x) + ancien\Lambda(x) * \Delta$
    end if
end for

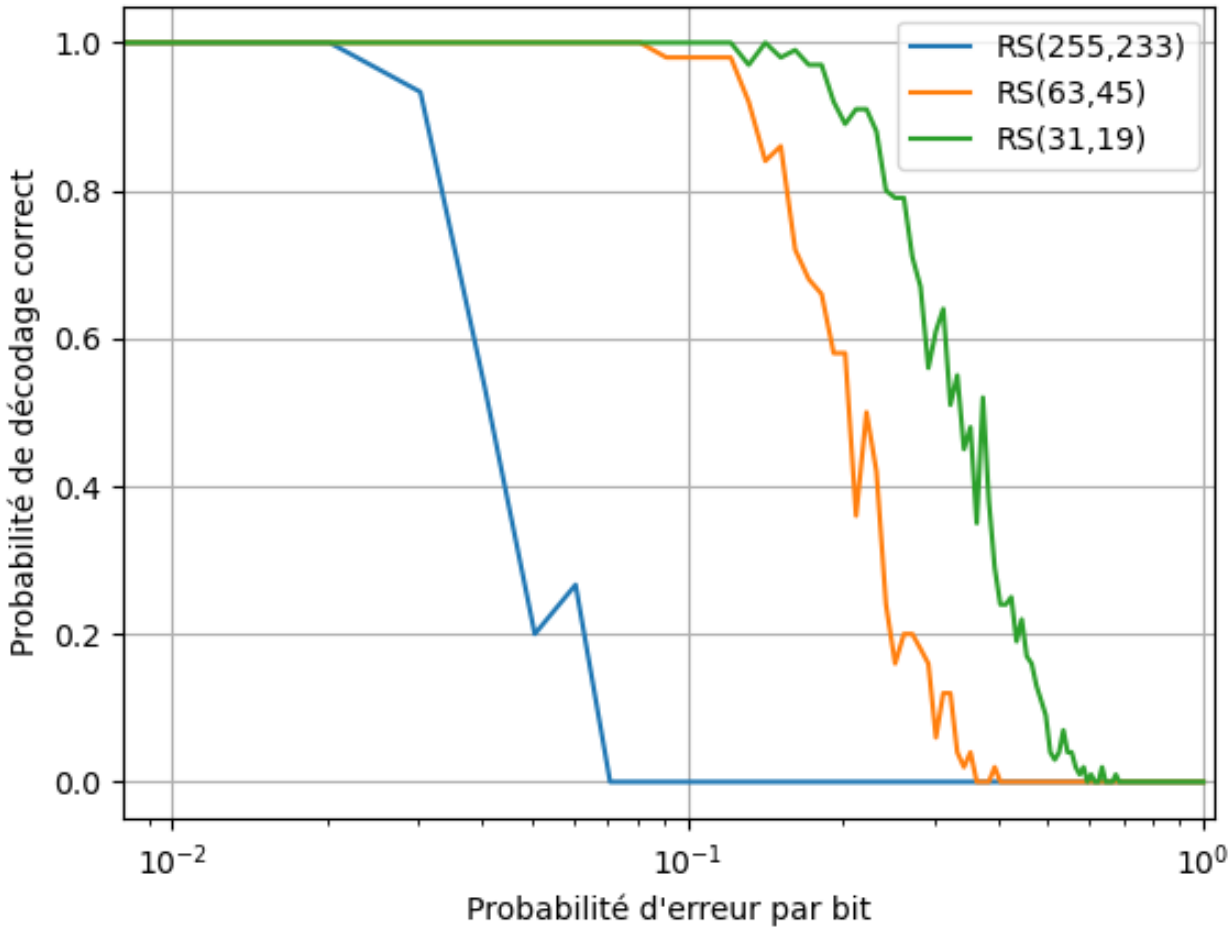
Formule de Forney

- $\forall j \in [[1, e]] : \dfrac{\Omega(\alpha^{-i_j})}{\Lambda'(\alpha^{-i_j})} = -E_{i_j}$

Résultats


Vérification de la correction de l'implémentation, n = 127
Probabilité de décodage correct
Taux de code : k/n
0.0
0.2
0.4
0.6
0.8
1.0
2 erreurs
4 erreurs
8 erreurs
16 erreurs
32 erreurs
64 erreurs

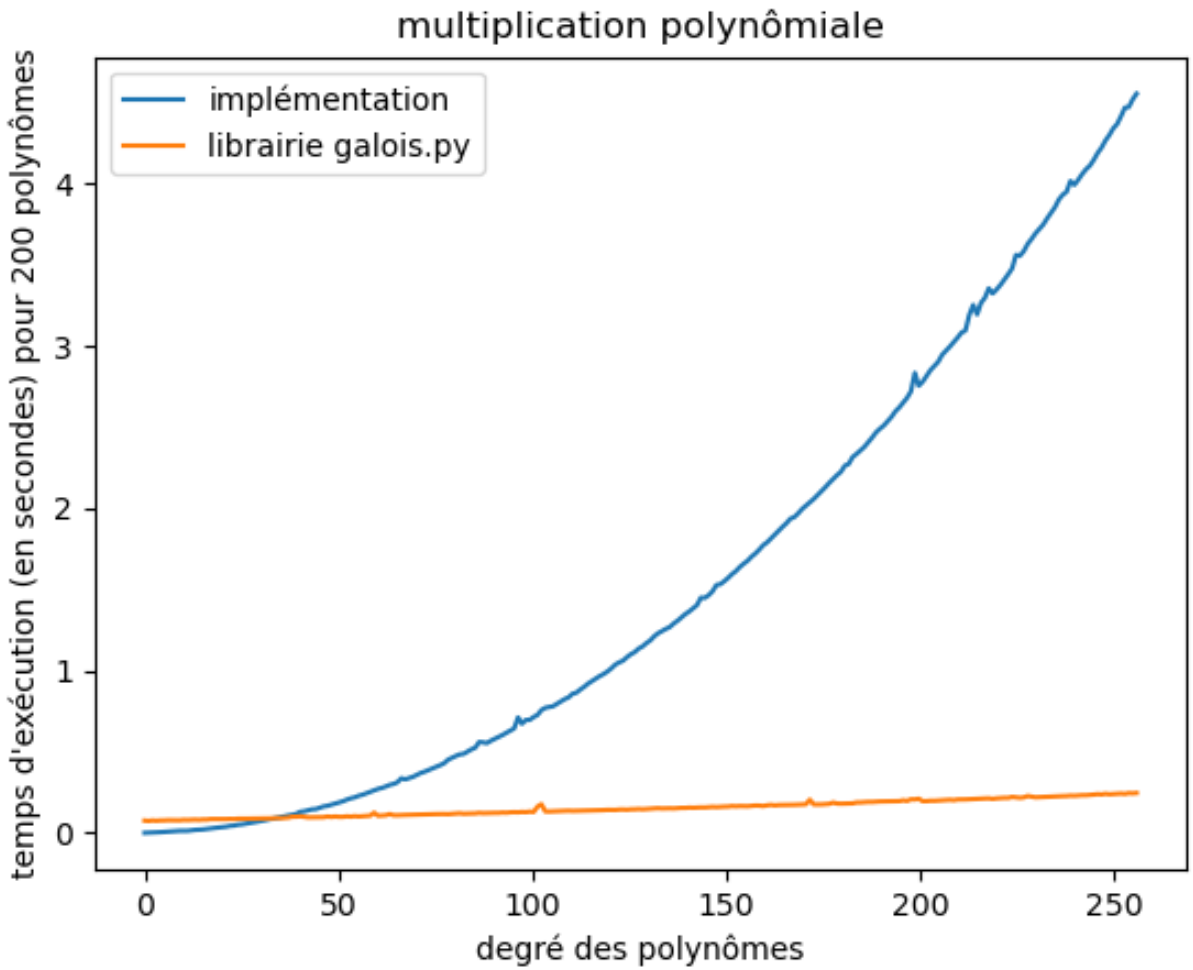
Résultats


RS(255,233)
RS(63,45)
RS(31,19)
Probabilité de décodage correct
1.0
0.8
0.6
0.4
0.2
0.0
10^{-2}
10^{-1}
10^{0}
Probabilité d'erreur par bit

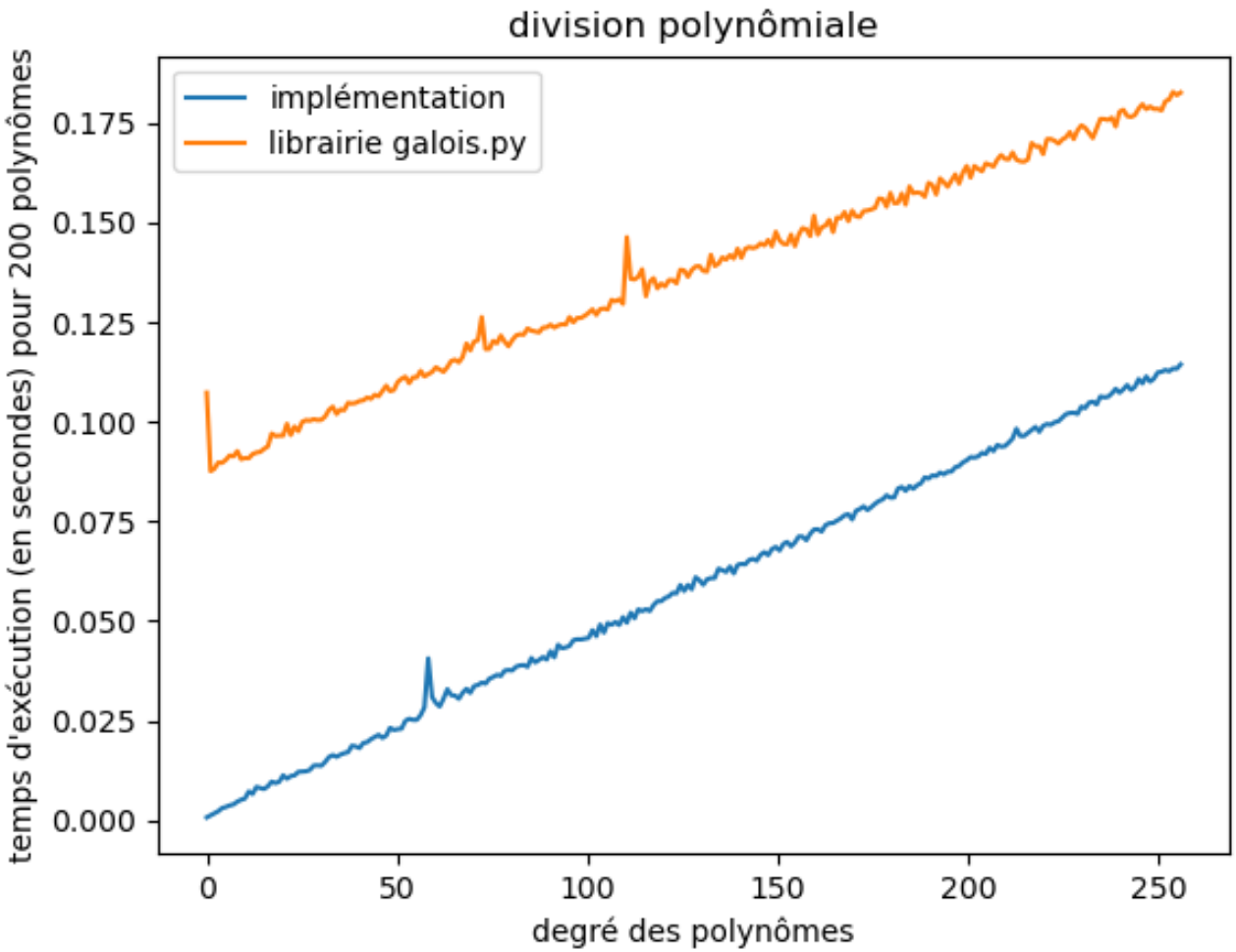
Conclusion

- Limites informatiques et physiques de cette étude

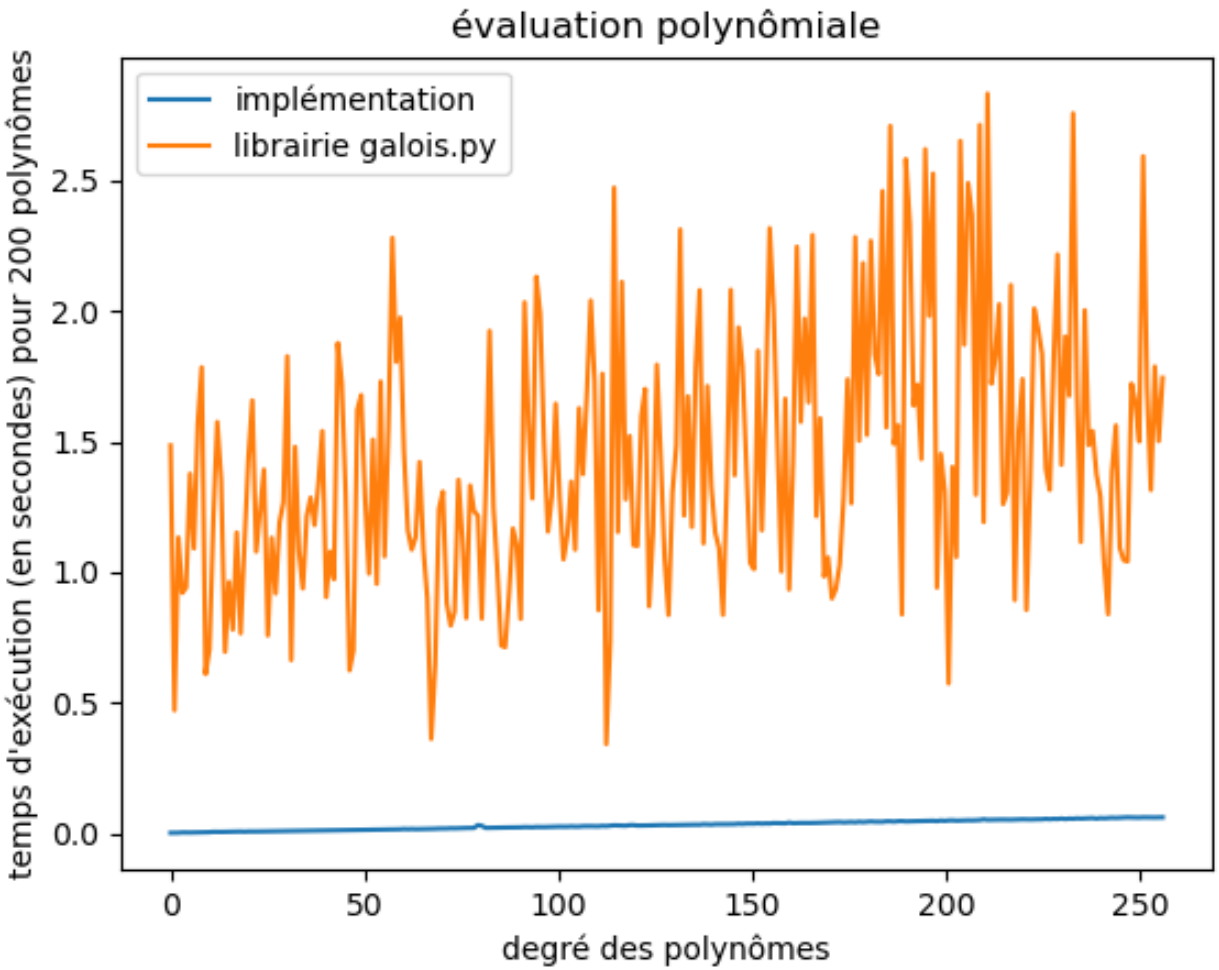
Résultats


multiplication polynômiale
implémentation
librairie galois.py
temps d'exécution (en secondes) pour 200 polynômes
degré des polynômes
0
1
2
3
4
0
50
100
150
200
250

Résultats


division polynômiale
implémentation
librairie galois.py
temps d'exécution (en secondes) pour 200 polynômes
0.000
0.025
0.050
0.075
0.100
0.125
0.150
0.175
0
50
100
150
200
250
degré des polynômes

Résultats


évaluation polynômiale
implémentation
librairie galois.py
temps d'exécution (en secondes) pour 200 polynômes
0.0
0.5
1.0
1.5
2.0
2.5
0
50
100
150
200
250
degré des polynômes

Améliorations possibles

- Performances : Utiliser Numpy, ou des librairies préconçues

```
def multiplication_reduite1(x,y,prim=0):
    def multiplication(x,y):
        z = 0
        i = 0
        while (y>>i) > 0:
            if y & (1<<i):
                z ^= x<<i
            i += 1
        return z
    def bitlength(n):
        b = 0
        while n>>b:
            b += 1
        return b
    def modular_reduction(dividende,diviseur):
        d1 = bitlength(dividende)
        d2 = bitlength(diviseur)
        if d1 < d2:
            return dividende
        for i in range(d1-d2, -1, -1):
            if dividende & (1 << i+d2-1):
                dividende ^= diviseur << i
        return dividende
    res = multiplication(x,y)
    if prim > 0:
        res = modular_reduction(res,prim)
    return int(res)
```

```
#initialisation des tables
prim = 0x11d
exp = [0] * 512
log = [0] * 256
x = 1
for i in range(0,255):
    exp[i] = x
    log[x] = i
    x = multiplication_reduite1(x,2,prim)
for i in range(255, 512):
    exp[i] = exp[i-255]
def mult(x,y):
    if x == 0 or y == 0:
        return 0
    return exp[log[x] + log[y]]
def division(x,y):
    if y == 0:
        raise ZeroDivisionError()
    if x == 0:
        return 0
    return exp[(log[x] + 255 - log[y] )%255]
def puissance(x, n):
    return exp[(log[x] * n)%255]
def inverse(x):
    return exp[255-log[x]]
```

```
def poly_par_scalaire(p,x):
    r = [0] * len(p)
    for i in range(0, len(p)):
        r[i] = mult(p[i], x)
    return r
def poly_somme(p,q):
    r = [0] * max(len(p),len(q))
    for i in range(0,len(p)):
        r[i+len(r)-len(p)] = p[i]
    for i in range(0,len(q)):
        r[i+len(r)-len(q)] ^= q[i]
    return r
def poly_soustraction(p,q):
    r = [0] * max(len(p),len(q))
    for i in range(0,len(p)):
        r[i+len(r)-len(p)] = p[i]
    for i in range(0,len(q)):
        r[i+len(r)-len(q)] ^= q[i]
    return r
def poly_produit(p,q):
    r = [0] *(len(p)+len(q)-1)
    for j in range(0,len(q)):
        for i in range(0,len(p)):
            r[i+j] ^=mult(p[i], q[j])
    return r
```

```
def poly_evaluation_horner(p,x):
    if p == []:
        return -1
    y = p[0]
    for i in range(1,len(p)):
        y = mult(y,x) ^ p[i]
    return y
def rs_generateur_poly(nsym):
    g = [1]
    for i in range(0, nsym):
        g = poly_produit(g, [1, puissance(2, i)])
    return g
def poly_division(dividende,diviseur):
    msg_out = list(dividende)
    for i in range(0, len(dividende) - (len(diviseur)-1)):
        coef = msg_out[i]
        if coef != 0:
            for j in range(1, len(diviseur)):
                if diviseur[j] != 0:
                    msg_out[i + j] ^= mult(diviseur[j], coef)
    separator = -(len(diviseur)-1)
    return msg_out[:separator], msg_out[separator:]
def encode_rs(message, nsym):
    gen = rs_generateur_poly(nsym)
    inutile, code_de_rs = poly_division(message + [0] * (len(gen)-1), gen)
    message_encode = message + code_de_rs
    return message_encode
```

```
def calcul_syndrome(message, nsym):
    syndrome = [0] * nsym
    for i in range(0,nsym):
        syndrome[i] = poly_evaluation_horner(message,puissance(2,i))
    return [0] + syndrome
def poly_localisation(position_erreur):
    localisateur = [1]
    for i in position_erreur :
        localisateur = poly_produit(localisateur,
            poly_somme([1], [puissance(2, i), 0]) )
    return localisateur
def trouver_poly_erreurs(syndromes,localisateur, nsym):
    reste = poly_produit(syndromes, localisateur)
    reste = reste[len(reste)-(nsym+1):]
    return reste
def corriger_erreurs(message, syndromes, err_pos):
    coef_pos = [len(message)-1-p for p in err_pos]
    err_loc = poly_localisation(coef_pos)
    err_eval = trouver_poly_erreurs(syndromes[::-1], err_loc,
        len(err_loc)-1)[::-1]
    X=[]
    for i in range(0,len(coef_pos)):
        l = 255 - coef_pos[i]
        X.append(puissance(2, -l))
    E = [0] * len(message)
    longueur_X = len(X)
    for i, Xi in enumerate(X):
        Xi_inv = inverse(Xi)
        err_loc_prime_tmp = []
        for j in range(0,longueur_X):
            if j != i:
```

```
        for coef in err_loc_prime_tmp:
            err_loc_prime = mult(err_loc_prime,coef)
        y = poly_evaluation_horner(err_eval[::-1], Xi_inv)
        y = mult(puissance(Xi, 1), y)
        if err_loc_prime == 0:
            return []
            # raise ReedSolomonError("Magnitude d'erreur introuvable")
        magnitude = division(y, err_loc_prime)
        E[err_pos[i]] = magnitude
    print(message,E)
    message = poly_somme(message,E)
    return message
def trouver_erreur_localisateur(syndromes, nsym):
    err_loc = [1]
    ancien_loc = [1]
    L = 0
    decalage_syndrome = len(syndromes) - nsym
    for i in range(0, nsym):
        r = i + decalage_syndrome
        delta = syndromes[r]
        for j in range(1,len(err_loc)):
            delta ^= mult(err_loc[-(j+1)], syndromes[r-j])
        ancien_loc = ancien_loc + [0]
        if delta != 0:
            if 2*L <= r:
                ancien_loc, err_loc = poly_par_scalaire(err_loc,
                    inverse(delta)), poly_par_scalaire(ancien_loc,delta)
                L = r-L
            err_loc = poly_somme(err_loc,
            poly_par_scalaire(ancien_loc, delta))
```

```
    while len(err_loc) and err_loc[0] == 0:
        del err_loc[0]
    erreurs = len(err_loc) - 1
    if (erreurs) * 2 > nsym:
        return []
        # raise ReedSolomonError("Trop d'erreurs     corriger")
    return err_loc
def trouver_erreurs(err_loc, nmess):
    erreurs = len(err_loc) - 1
    err_pos = []
    for i in range(nmess):
        if poly_evaluation_horner(err_loc, puissance(2,i))==0:
            err_pos.append(nmess-1-i)
    if len(err_pos) != erreurs:
        return []
        # raise ReedSolomonError("Nombre d'erreurs trouv es incorrect")
    return err_pos
def corriger_message(message, nsym):
    if len(message) > 255:
        return [],[]
        # raise ValueError("Message trop long")
    message_2 = list(message)
    syndromes = calcul_syndrome(message_2, nsym)
    if max(syndromes) == 0:
        return message_2[:-nsym], message_2[-nsym:]
    err_loc = trouver_erreur_localisateur(syndromes,nsym)
    err_pos = trouver_erreurs(err_loc[::-1], len(message_2))
    if err_pos is None:
        return [],[]
        # raise ReedSolomonError("Erreur n'a pas pu   tre   trouv e")
```

```
message_2 = corriger_erreurs(message_2, syndromes, err_pos)
syndromes = calcul_syndrome(message_2, nsym)
if max(syndromes) > 0:
    return [],[]
    # raise ReedSolomonError("Message n'a pas pu  tre  corrig ")
return message_2[:-nsym], message_2[-nsym:]
```